From Counterexamples to Inductive Proofs

A Tutorial on SBV

ANONYMOUS AUTHOR(S)

SBV is a Haskell library for reasoning about programs with SMT solvers. This tutorial is a guided path through it for Haskell programmers with no prior background in automated reasoning. We begin with one-line theorems and counterexamples, write specifications that are independent of the code they check, search for values that satisfy constraints, and program a live solver session from Haskell. We then define symbolic algebraic datatypes and recursive functions, and prove properties that no solver establishes on its own by supplying induction and equational reasoning as ordinary Haskell code. A detailed proof of Kadane's maximum-segment-sum algorithm shows how to find the right invariant, and an overview of a verified tautology checker shows how such proofs scale. Finally, we compile a verified function to C. Along the way we explain the boundary between Haskell evaluation and symbolic reasoning, which is the source of most surprises for new users. Every listing is extracted from a companion program that is checked against the current release.

Additional Key Words and Phrases: Haskell, SMT, symbolic programming, theorem proving, induction, tutorial

1 INTRODUCTION

Haskell programmers already state properties of their code: QuickCheck properties [Claessen and Hughes 2000] are ordinary functions that return `Bool`, and a test campaign searches for inputs that falsify them. SBV [Erkök 2026] lets the same programmer ask a stronger question. A property over *symbolic* values is translated to logic and handed to an SMT solver, which either proves it for every input or produces a concrete counterexample:

```
ghci> prove $ \x -> abs (x :: SInteger) .>= 0
Q.E.D.
ghci> prove $ \x -> abs (x :: SInt8) .>= 0
Falsifiable. Counter-example:
  s0 = -128 :: Int8
```

The two properties differ only in a type. Over mathematical integers, absolute values are nonnegative. Over signed bytes, 128 is not representable, so `abs (-128)` is −128. The solver did not test this value by luck: it searched all 256 inputs symbolically and reported the one that fails.

This tutorial is for Haskell programmers who want to use solvers in that way and then go further. It assumes familiarity with type classes and monads but no knowledge of SMT, logic, or theorem proving. Each section introduces one idea and builds on the previous ones:

- Sections 2–4 cover proofs, counterexamples, specifications, and searches for values satisfying constraints.
- Section 5 explains the boundary between Haskell evaluation and symbolic reasoning, which governs how recursion and data structures behave.
- Sections 6–7 program a live solver session and introduce symbolic algebraic datatypes.
- Sections 8–10 prove properties of recursive functions with SBV's theorem-proving layer, TP, ending with a detailed proof of Kadane's algorithm and an overview of a verified tautology checker.
- Section 11 compiles a verified function to C, and Section 12 explains enough of the implementation to make SBV's behavior predictable.

2026. XXXX-XXXX/2026/11-ART $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Section 13 collects the pitfalls in one place, and Section 14 relates SBV to other tools.

Following along. SBV is on Hackage; add `sbv` to a project's dependencies and make an SMT solver available on the `PATH`. Z3 [de Moura and Bjørner 2008] is the default. A few examples below use CVC5, and SBV also supports Yices, Bitwuzla, Boolector, MathSAT, ABC, OpenSMT, and dReal. Most examples need only `import Data.SBV`; the solver programming interface is in `Data.SBV.Control`, list operations in `Data.SBV.List`, and the proof layer in `Data.SBV.TP`. The supplementary material contains a single program from which every listing in this paper is extracted. It runs each example, checks its result independently where that is possible, and also contains solutions to the exercises.

2 FIRST PROOFS AND COUNTEREXAMPLES

2.1 Symbolic values and their types

SBV's central type is `SBV a`, a symbolic value whose concrete counterpart has type `a`. Abbreviations name the common cases: `SBool`, `SInteger` (unbounded integers), `SWord8` through `SWord64` and `SInt8` through `SInt64` (bit-vectors with wrap-around arithmetic), `SReal` (exact real numbers), `SFloat` and `SDouble` (IEEE-754), `SString`, and `SList a`. There are also arrays, sets, tuples, `Maybe`, `Either`, and user-defined algebraic datatypes.

These types are not interchangeable encodings of one numeric type; each is a different mathematical problem. The introduction showed that the choice between `SInteger` and `SInt8` changes a theorem into a false statement. Floating point has its own surprises:

```
ghci> prove $ \(x :: SFloat) -> x .== x
Falsifiable. Counter-example:
  s0 = NaN :: Float
```

Equality in IEEE-754 is not reflexive, and the solver knows it. Choosing a symbolic type is choosing the semantics the solver will use, so it deserves the same care as choosing a type for a production program.

2.2 Operators with a dot

Arithmetic on symbolic values uses the ordinary `Num` class, so `x + 1` and `2 * y` work as expected. Comparisons cannot reuse `Eq` and `Ord`: their methods return a Haskell `Bool`, which must be known when Haskell evaluates it, while the result of comparing symbolic values may be unknown. SBV therefore provides `.==`, `./=`, `.<`, `.<=`, `.>`, and `.>=`, which return `SBool`. Boolean connectives are `.&&`, `.||`, `sNot`, and implication `.=>`.

For the same reason, Haskell's `if-then-else` cannot branch on an `SBool`. The function `ite :: Mergeable a => SBool -> a -> a -> a` builds a symbolic choice that the solver resolves. This distinction between *Haskell control* and *symbolic control* recurs throughout the tutorial. When all arguments are known, symbolic operations simply compute: `ite (3 .> 2) 10 20` evaluates to the literal `10` without involving a solver.

2.3 What prove does

Calling `prove` on a function of symbolic arguments creates a fresh unknown for each argument, evaluates the function to obtain a symbolic formula $P(\vec{x})$, and asks the solver whether $\neg P(\vec{x})$ is satisfiable. If it is not, P holds for every input and SBV prints `Q.E.D.` If it is, the satisfying assignment is a counterexample. A third outcome is possible: for undecidable or very hard problems, the solver may give up or time out. SBV reports this as an unknown result, which is neither a proof nor a refutation.

Table 1. Entry points into SBV. All of them consume the same symbolic programs.

Interface	You supply	You get back	Section
`prove`	A property	Proof, counterexample, or unknown	2–3
`sat`, `allSat`	Constraints on unknowns	One or all satisfying models	4
`optimize`	Constraints and objectives	An optimal model	4
`query`	A Haskell program driving the solver	Whatever the program computes	6
TP	Lemmas, induction, calculations	Checked proofs with dependencies	8–10
`compileToC`	A symbolic function	C source code	11

Unknowns can also be created by name inside the `Symbolic` monad, which gives readable counterexamples and allows assumptions:

```
prove $ do x <- sInteger "x"
           y <- sInteger "y"
           constrain $ x .> 0 .&& y .> 0
           pure $ x * y .> 0
```

Here `constrain` restricts attention to inputs that satisfy the assumption. A contradictory assumption makes every property provable, so assumptions should be kept as weak as the property permits.

Table 1 lists the main ways of using a symbolic model. The rest of the tutorial visits each of them.

3 WRITING SPECIFICATIONS

A proof is only as good as the property it proves. The most useful properties compare an implementation with a specification written in a different, simpler way. Consider addition on unsigned bytes that saturates at 255 instead of wrapping. An implementation can avoid overflow by testing before it adds, while the specification can use unbounded integers, where overflow cannot happen:

```
satAdd :: SWord8 -> SWord8 -> SWord8
satAdd x y = ite (x .> 255 - y) 255 (x + y)

satAddSpec :: SWord8 -> SWord8 -> SBool
satAddSpec x y = sFromIntegral (satAdd x y) .== ite (total .> 255) 255 total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

The guard `x .> 255 - y` cannot overflow, because `255 - y` is representable for every byte `y`, and on the other branch `x + y` does not wrap. In the specification, `sFromIntegral` converts each byte to an integer *before* adding. Converting the already wrapped byte sum would repeat the implementation's arithmetic and could hide its mistakes.

```
ghci> prove satAddSpec
Q.E.D.
```

The proof covers all 65,536 input pairs. A tempting alternative implementation shows why the independent specification matters:

```
wrongAdd :: SWord8 -> SWord8 -> SWord8
wrongAdd x y = ite (x + y .> 255) 255 (x + y)

wrongAddSpec :: SWord8 -> SWord8 -> SBool
wrongAddSpec x y = sFromIntegral (wrongAdd x y) .== ite (total .> 255) 255 total
  where total = sFromIntegral x + sFromIntegral y :: SInteger
```

```
ghci> prove $ do { x <- sWord8 "x"; y <- sWord8 "y"; pure (wrongAddSpec x y) }
Falsifiable. Counter-example:
  x = 223 :: Word8
  y = 124 :: Word8
```

The sum $223 + 124 = 347$ wraps to 91, and no byte is greater than 255, so the saturation branch is never taken. Had the specification also used byte arithmetic, it would have agreed with the bug. Your solver may report a different pair; any pair whose sum exceeds 255 is a valid counterexample, and the companion checks the reported pair with ordinary Haskell arithmetic.

Exercise 1. Write `satSub :: SWord8 -> SWord8 -> SWord8`, subtraction that saturates at 0, together with a specification over `SInteger`, and prove that they agree.

4 FINDING VALUES

Proving a property asks whether every input satisfies it. The dual question asks whether *some* input satisfies a constraint, and which one. The function `sat` returns one satisfying model, and `allSat` enumerates all of them. Which bytes are unchanged by squaring?

```
idempotentBytes :: IO AllSatResult
idempotentBytes = allSat $ \(x :: SWord8) -> x * x .== x
```

```
ghci> idempotentBytes
Solution #1:
  s0 = 1 :: Word8
Solution #2:
  s0 = 0 :: Word8
Found 2 different solutions.
```

The answer is a small theorem in disguise: $x(x-1) \equiv 0 \pmod{256}$, and since x and $x-1$ are coprime, one of them must be divisible by 256.

When some solutions are better than others, `optimize` finds the best one. A workshop makes two products with profits 2 and 3, subject to two resource limits:

```
production :: IO OptimizeResult
production = optimize Lexicographic $ do
  x <- sInteger "x"
  y <- sInteger "y"
  constrain $ x .>= 0 .&& y .>= 0
  constrain $ x + 2 * y .<= 10
  constrain $ 3 * x + y .<= 15
  maximize "profit" (2 * x + 3 * y)
```

```
ghci> production
```

```
Optimal model:
  x      =  4 :: Integer
  y      =  3 :: Integer
  profit = 17 :: Integer
```

With several objectives, the first argument of `optimize` selects how to combine them: lexicographically, independently, or as a Pareto front. Over `SInteger` this is integer linear programming; replacing the types with `SReal` gives linear programming over the rationals.

5 HASKELL STRUCTURE AND SYMBOLIC STRUCTURE

Every SBV program runs in two stages. First, Haskell evaluates the program and builds a symbolic formula. Second, the solver reasons about that formula. Knowing which stage handles which part of a program explains most of SBV's behavior.

5.1 Haskell lists of symbolic values

A Haskell list `[SInteger]` has a spine that Haskell owns and elements that may be symbolic. Functions such as `sum`, `map`, and `foldr` work on it unchanged, because they only rearrange the spine and apply `Num` operations to the elements:

```
nonNegSum :: Int -> Predicate
nonNegSum n = do
  xs <- mapM (\i -> sInteger ("x" ++ show i)) [1 .. n]
  pure $ sAll (.>= 0) xs .=> sum xs .>= 0
```

For any fixed `n`, `prove (nonNegSum 5)` succeeds immediately. Haskell unfolds `sum` into `x1 + x2 + x3 + x4 + x5` before the solver sees anything; the recursion has disappeared. This style is ideal for fixed-size problems: circuits, unrolled loops, puzzles on fixed boards. The price is that the theorem is about lists of length five, not all lists.

5.2 Symbolic lists and recursive functions

The type `SList Integer` is a list whose length and contents are both unknown. To state a property of *every* list, we need a function whose recursion is part of the logic, not something Haskell unfolds. Writing a naive recursive Haskell function over `SList` does not work: whether the list is empty is unknown, so Haskell would unfold both branches forever. SBV's `smtFunction` solves this problem. It gives the function a name, and recursive calls become calls to that named function in the logic:

```
sumL :: SList Integer -> SInteger
sumL = smtFunction "sumL" $ \xs -> [sCase| xs of
          []     -> 0
          y : ys -> y + sumL ys
      |]
```

The `sCase` quasi-quoter provides pattern matching on symbolic values. The definition reads like ordinary Haskell, but SBV translates it to a single recursive SMT-LIB definition [Barrett et al. 2025]. Because `sumL` is a function of symbolic values, it still computes on literals: `sumL [1,2,3]` evaluates to `6` without a solver. SBV also checks that recursive definitions terminate; for `sumL`, it finds a suitable measure automatically.

Now try the obvious generalization of `nonNegSum`:

```
prove $ \xs -> SL.all (.>= 0) xs .=> sumL xs .>= 0
```

The property is true, but proving it requires induction on the list, and SMT solvers do not perform induction. Typically the solver runs until it is stopped or reports an unknown result. Section 8 shows how to supply the induction ourselves.

The general rule is: *recursion over Haskell structure happens while building the problem; recursion through* `smtFunction` *becomes part of the problem.* The former yields bounded but fully automatic results; the latter yields unbounded results that may need a proof.

6 PROGRAMMING THE SOLVER

The functions `prove`, `sat`, and `optimize` each run a single solver interaction. Many applications need a conversation: ask, inspect the answer, and ask a refined question. SBV's `query` mode hands control of a live solver session to a Haskell program. The following function finds the smallest integer greater than 10 that leaves remainder 3 when divided by 7 and remainder 4 when divided by 5:

```
smallest :: IO Integer
smallest = runSMT $ do
  x <- sInteger "x"
  constrain $ x .> 10
  constrain $ x `sMod` 7 .== 3
  constrain $ x `sMod` 5 .== 4
  query $ let descend best = do
                r <- checkSat
                case r of
                  Sat   -> do v <- getValue x
                              constrain $ x .< literal v
                              descend (Just v)
                  Unsat -> maybe (io $ fail "no solution") pure best
                  _     -> io $ fail "inconclusive"
          in descend Nothing
```

```
ghci> smallest
24
```

The constraints before `query` set up the problem as usual. Inside the query, `checkSat` asks for a model, and `getValue` reads the value of `x` in that model. That value is an ordinary `Integer`, so Haskell can branch on it and use it to build the next constraint, `x .< literal v`, which asks for a smaller solution. When no smaller solution exists, the last model found is the minimum. The `case` on the solver's answer is Haskell control; the constraint that it adds is symbolic. Keeping the two apart is what makes the solver usable as a component of a larger algorithm.

Three properties of this loop are worth noting. First, it terminates only because the constraints admit a least solution; the same loop over a set without a minimum would run forever, even though each call to the solver returns. The termination argument belongs to the client algorithm, not to the solver. Second, the solver may return the candidates in any order. Our loop does not depend on that order; it only relies on unsatisfiability at the end. Third, the loop handles an unknown answer separately: an inconclusive solver response is not evidence that no smaller value exists.

Queries can do much more: `push` and `pop` add and retract temporary assumptions, named constraints allow unsatisfiable cores to be extracted, and new unknowns can be created in the middle of a session with `freshVar`. The last point is less trivial than it looks: a new variable may have a type, such as a user-defined datatype, that the solver has not seen yet. SBV tracks the declarations

each new term needs and sends them before the term is used, so queries compose without regard to the order in which types are first used.

Exercise 2. Change `smallest` to find the *largest* solution below 100. (The answer is 94.)

7 SYMBOLIC DATATYPES

User-defined algebraic datatypes become symbolic with one line of Template Haskell. Here is a tiny expression language and its evaluator:

```
data Expr = Num Integer
          | Add Expr Expr
          | Mul Expr Expr
          deriving Show
mkSymbolic [''Expr]

eval :: SExpr -> SInteger
eval = smtFunction "eval" $ \e -> [sCase| e of
          Num n   -> n
          Add a b -> eval a + eval b
          Mul a b -> eval a * eval b
       |]

puzzle :: IO SatResult
puzzle = sat $ \e -> eval e .== 42 .&& isMul e .&& sNot (isNum (getMul_1 e))
```

The call `mkSymbolic [''Expr]` generates the type `SExpr`, symbolic constructors (`sNum`, `sAdd`, `sMul`), recognizers (`isNum`, `isAdd`, `isMul`), and field accessors such as `getMul_1`. The evaluator is an `smtFunction`, so it is a recursive definition in the logic, and `sCase` matches on an expression whose constructor may be unknown. The puzzle asks the solver to *synthesize* an expression: a multiplication whose left argument is not a number, evaluating to 42.

```
ghci> puzzle
Satisfiable. Model:
  s0 = Mul (Mul (Num 1) (Num (-1))) (Num (-42)) :: Expr
```

The model is an ordinary Haskell value of type `Expr`, which we can inspect, print, or evaluate with a separate Haskell interpreter. The companion does exactly that to check the answer independently of the solver. Different solver versions may find different expressions.

The same datatype can be the subject of proofs: a property of `eval` over all expressions is a statement about an unbounded, recursive structure, and will need induction. That is the topic of the next section.

8 PROOFS BY INDUCTION

SMT solvers decide many theories well, but they do not discover induction arguments. SBV's TP layer (in `Data.SBV.TP`) lets the programmer write the argument in Haskell and has the solver check each step. The programmer decides *what* to prove and *how* to break it into steps; the solver decides whether each step is valid.

8.1 A first inductive proof

We return to the property that eluded `prove` in Section 5:

```
sumNonNeg :: TP (Proof (Forall "xs" [Integer] -> SBool))
sumNonNeg = induct "sumNonNeg"
  (\(Forall xs) -> SL.all (.>= 0) xs .=> sumL xs .>= 0) $
  \ih (x, xs) -> [SL.all (.>= 0) (x .: xs)]
              |- sumL (x .: xs) .>= 0
              =: x + sumL xs .>= 0
              ?? ih
              =: sTrue
              =: qed
```

The type `Proof (Forall "xs" [Integer] -> SBool)` records the statement of the theorem: for every integer list `xs`, a property holds. The name in `Forall` documents the quantified variable and is used in output and instantiation. The function `induct` applies list induction: SBV checks the base case (the empty list) automatically and gives the user the inductive step. In that step, `ih` is the induction hypothesis, a proof of the property for `xs`, and the goal is to show it for `x .: xs`.

The step is a *calculation*. Its first line lists the assumptions, here the precondition for the longer list, and `|-` starts a chain of expressions connected by `=:`. Each link must be justified by the definitions, the assumptions, and any hints supplied with `??`. The chain ends with `qed`. Running the proof shows each checked step:

```
ghci> runTP sumNonNeg
Inductive lemma: sumNonNeg
  Step: Base                 Q.E.D.
  Step: 1                    Q.E.D.
  Step: 2                    Q.E.D.
  Result:                    Q.E.D.
Functions proven terminating: sbv.foldr, sumL
```

Step 1 unfolds `sumL` once. Step 2 uses the hypothesis: `x` is nonnegative by assumption and so is `sumL xs`, so the sum is too. The `Result` line checks that the chain, together with the base case, establishes the theorem as stated. The last line reports that the recursive functions used, including the library's `foldr` that underlies `SL.all`, were checked for termination.

8.2 What the solver checks

A calculation $e_0 = e_1 = \cdots = e_n$ under assumptions A, with hints H_i attached to step i, generates one solver query per step:

$$A \wedge H_i \wedge \neg(e_i = e_{i+1}) \quad \text{must be unsatisfiable.}$$

Each query is small and local. This is why calculations work: the solver never has to find the overall argument, only to confirm that consecutive expressions are equal. If a step fails, SBV reports which one, which turns proof debugging into a question about one equation. A step that is too large can be split by adding intermediate expressions; a step that fails although it is true usually needs a hint.

8.3 Generalizing the statement

Reversing a list with an accumulator is a standard optimization:

```
revAcc :: SList Integer -> SList Integer -> SList Integer
revAcc = smtFunction "revAcc" $ \acc xs -> [sCase| xs of
            []     -> acc
            x : xt -> revAcc (x .: acc) xt
         |]
```

We expect `revAcc [] xs` to equal `SL.reverse xs`. The direct attempt is instructive:

```
revWeak :: TP (Proof (Forall "xs" [Integer] -> SBool))
revWeak = induct "revWeak"
  (\(Forall xs) -> revAcc [] xs .== SL.reverse xs) $
  \ih (x, xs) -> [] |- revAcc [] (x .: xs)
                    =: revAcc [x] xs
                    ?? ih
                    =: SL.reverse xs SL.++ [x]
                    =: SL.reverse (x .: xs)
                    =: qed
```

```
ghci> runTP revWeak
Inductive lemma: revWeak
  Step: Base                Q.E.D.
  Step: 1                   Q.E.D.
  Step: 2
*** Failed to prove revWeak.2.
```

Step 2 tries to use the induction hypothesis, but the hypothesis is about `revAcc [] xs`, while the expression contains `revAcc [x] xs`. After one step of recursion, the accumulator is no longer empty, and the hypothesis says nothing about it. (We ran this with a five-second timeout; without one, the solver searches until it is interrupted.)

The fix is the oldest trick in inductive proofs: prove a more general statement. The theorem must describe `revAcc` for *every* accumulator, so that the hypothesis can be used at whatever accumulator the recursion produces:

```
revAccCorrect :: TP (Proof (Forall "xs" [Integer] -> Forall "acc" [Integer] -> SBool))
revAccCorrect = induct "revAccCorrect"
  (\(Forall xs) (Forall acc) -> revAcc acc xs .== SL.reverse xs SL.++ acc) $
  \ih (x, xs) acc -> [] |- revAcc acc (x .: xs)
                        =: revAcc (x .: acc) xs
                        ?? ih
                        =: SL.reverse xs SL.++ (x .: acc)
                        =: (SL.reverse xs SL.++ [x]) SL.++ acc
                        =: SL.reverse (x .: xs) SL.++ acc
                        =: qed
```

Here `acc` is universally quantified, and the induction hypothesis `ih` holds for all accumulators. When it is supplied as a hint, the solver instantiates it with `x .: acc`. The remaining steps use ordinary laws of `++` and `reverse`, which the solver knows. The original statement is the special case `acc = []`, which a one-line `lemma` can derive.

Exercise 3. Prove that `SL.length (revAcc acc xs) .== SL.length xs + SL.length acc`. Does the statement need to be generalized, and why?

9 CASE STUDY: KADANE'S ALGORITHM

The maximum segment sum problem asks for the largest sum of a contiguous segment of a list, where the empty segment has sum 0. For `[1, -2, 3, 4, -1, 2]`, the answer is 8, from `[3, 4, -1, 2]`. Kadane's algorithm [Bentley 1984] solves it in a single left-to-right pass. Its correctness is not obvious, which makes it a good example of finding and proving an invariant.

9.1 Specification

We specify the problem by its most direct recursive reading:

```
mss :: SList Integer -> SInteger
mss = smtFunction "mss" $ \xs -> [sCase| xs of
        []    -> 0
        _ : t -> mssBegin xs `smax` mss t
     |]

mssBegin :: SList Integer -> SInteger
mssBegin = smtFunction "mssBegin" $ \xs -> [sCase| xs of
             []    -> 0
             h : t -> 0 `smax` (h `smax` (h + mssBegin t))
          |]
```

`mssBegin xs` is the best sum of a segment that starts at the beginning of `xs`: either the empty segment (0), just the head, or the head followed by the best segment starting at the tail. `mss xs` is the best of `mssBegin xs` and the best segment of the tail. These definitions are clearly correct and clearly inefficient: they take quadratic time. Since they are symbolic functions, they also run on literal lists, which is a convenient way to test a specification before reasoning about it: `mss [1, -2, 3, 4, -1, 2]` evaluates to 8.

9.2 Implementation

Kadane's algorithm maintains two accumulators while it scans the list: `meh`, the maximum sum of a segment ending at the current position, and `msf`, the maximum found so far:

```
kadane :: SList Integer -> SInteger
kadane xs = kadaneHelper xs 0 0

kadaneHelper :: SList Integer -> SInteger -> SInteger -> SInteger
kadaneHelper = smtFunction "kadaneHelper" $ \xs meh msf -> [sCase| xs of
                 []    -> msf
                 h : t -> let meh' = 0 `smax` (h + meh)
                              msf' = msf `smax` meh'
                          in kadaneHelper t meh' msf'
               |]
```

At each element `h`, the best segment ending here either extends the best segment ending at the previous position or is empty, so the new `meh` is `0 `smax` (h + meh)`. The best so far is updated if that value is larger.

9.3 Finding the invariant

As with reversal, the theorem we want, `kadane xs .== mss xs`, concerns the initial accumulators `0` and `0`. An inductive proof will reach `kadaneHelper` with other accumulator values, so we must describe what `kadaneHelper xs meh msf` computes for *any* sensible `meh` and `msf`.

Think of the list as split into a prefix that has been processed and a suffix `xs` that has not. The best segment of the whole list lies in one of three places:

(1) entirely within the prefix: its sum is `msf`;
(2) entirely within the suffix: its sum is `mss xs`;
(3) across the boundary: it is a segment ending at the boundary, whose best sum is `meh`, followed by a segment starting at the boundary, whose best sum is `mssBegin xs`.

So the invariant is

$$kadaneHelper(xs, meh, msf) = \max\big(msf,\ mss(xs),\ meh + mssBegin(xs)\big),$$

under the preconditions that make the accumulators meaningful: $meh \geq 0$ (the empty segment is always available) and $msf \geq meh$ (the best segment so far is at least as good as the best one ending here). Without the preconditions the statement is false: for an empty `xs`, the right side would include *meh* even when it exceeds *msf*.

9.4 The proof

Once the invariant is written down, the proof is short:

```
kadaneCorrect :: TP (Proof (Forall "xs" [Integer] -> SBool))
kadaneCorrect = do
  inv <- induct "kadaneInvariant"
           (\(Forall xs) (Forall meh) (Forall msf) ->
                    (meh .>= 0 .&& msf .>= meh)
                .=> kadaneHelper xs meh msf
                    .== msf `smax` mss xs `smax` (meh + mssBegin xs)) $
           \ih (a, as) meh msf ->
              [meh .>= 0, msf .>= meh]
           |- let meh' = 0 `smax` (a + meh)
                  msf' = msf `smax` meh'
              in kadaneHelper (a .: as) meh msf
              =: kadaneHelper as meh' msf'
              ?? ih `at` (Inst @"meh" meh', Inst @"msf" msf')
              =: msf' `smax` mss as `smax` (meh' + mssBegin as)
              =: qed

  calc "kadaneCorrect" (\(Forall xs) -> kadane xs .== mss xs) $
       \xs -> let z = 0 :: SInteger
              in [] |- kadane xs
                    =: kadaneHelper xs z z
                    ?? inv `at` (Inst @"xs" xs, Inst @"meh" z, Inst @"msf" z)
                    =: z `smax` mss xs `smax` (z + mssBegin xs)
                    =: mss xs `smax` mssBegin xs
                    =: mss xs
                    =: qed
```

The inductive step unfolds `kadaneHelper` once and applies the hypothesis at the new accumulators. Here the hint is more specific than in the earlier proofs: `ih `at` (Inst @"meh" meh', Inst @"msf" msf')` instantiates the quantified accumulators explicitly. The solver could search for the instantiation, but stating it makes the step easier and documents the argument. The remaining equality between the instantiated hypothesis and the invariant for `a .: as` is arithmetic over `smax`, which the solver checks without further help, using the definitions of `mss` and `mssBegin`.

The preconditions are needed to instantiate the hypothesis: the new accumulators satisfy them because `meh'` is a maximum with 0 and `msf'` is a maximum with `meh'`.

The main theorem then follows by a calculation without induction. With both accumulators 0, the invariant gives $\max(0, mss(xs), mssBegin(xs))$, and the solver discharges the last two steps: `mss` is nonnegative and at least `mssBegin` by its definition.

```
ghci> runTP kadaneCorrect
Inductive lemma: kadaneInvariant
  Step: Base                          Q.E.D.
  Step: 1                             Q.E.D.
  Step: 2                             Q.E.D.
  Result:                             Q.E.D.
Lemma: kadaneCorrect
  Step: 1                             Q.E.D.
  Step: 2                             Q.E.D.
  Step: 3                             Q.E.D.
  Step: 4                             Q.E.D.
  Result:                             Q.E.D.
Functions proven terminating: kadaneHelper, mss, mssBegin
```

The whole proof takes about a tenth of a second with Z3.

9.5 Lessons

The effort in this proof went into one place: the invariant. Once we had explained *why* the algorithm works, as the three-way case analysis on where the best segment lies, the mechanical checking was immediate. This division of labor is typical of TP. The solver handles unfolding definitions, arithmetic, and case analysis on `smax`; the human supplies generalizations and intermediate statements. When a proof gets stuck, the useful question is rarely "which tactic is missing?" and usually "what is the statement that explains this step?"

10 CASE STUDY OVERVIEW: A VERIFIED TAUTOLOGY CHECKER

Larger developments are built from the same ingredients. SBV's documentation includes a verified tautology checker for propositional formulas in if-then-else form, after a classic development by Boyer and Moore [1979] that was later formalized in Imandra [Passmore 2025]. We sketch its structure rather than its proofs; the full development is in the module `Documentation.SBV.Examples.TP.TautologyChecker`. The listings in this section are quoted from that module.

Formulas and normalization. A formula is a constant, a variable, or a conditional:

```
data Formula = FTrue
             | FFalse
             | Var { fVar   :: Integer }
             | If  { ifCond :: Formula
                   , ifThen :: Formula
                   , ifElse :: Formula
                   }
```

The checker first normalizes a formula so that no conditional appears in the condition position of another conditional, by repeatedly rewriting `If (If p q r) l r'` to `If p (If q l r') (If r l r')`. Abbreviated slightly, the definition is:

```
normalize :: SFormula -> SFormula
```

```
normalize = smtFunctionWithMeasure "normalize"
    ( \f -> tuple (ifComplexity f, ifDepth f)
    , [measureLemma ifDepthNonNeg, measureLemma ifComplexityPos, ...] )
  $ \f -> [sCase| f of
            If (If p q r) l r' -> normalize (sIf p (sIf q l r') (sIf r l r'))
            If c l r           -> sIf c (normalize l) (normalize r)
            _                  -> f
          |]
```

This function is the first new ingredient. Its termination is not structural: the rewrite in the first branch produces a *larger* formula. Instead of letting SBV guess a measure, the definition supplies one through `smtFunctionWithMeasure`: a pair ordered lexicographically: a complexity measure that the rewrite does not increase, and the depth of conditionals in condition position, which it decreases. Each `measureLemma` is a proved lemma that SBV uses when checking the measure; for example, that the complexity of a formula is positive.

Checking. On a normalized formula, the checker walks down the conditionals, recording an assumed value for each variable the first time it is tested. When it meets a variable that was already assigned, it follows the corresponding branch; otherwise it checks both branches, once under each assumption. The function `isTautology f` normalizes `f` and runs this walk with no assumptions.

The theorems. Soundness says that a formula accepted by the checker evaluates to true under every assignment:

```
soundness :: TP (Proof (Forall "f" Formula -> Forall "bindings" [Binding] -> SBool))
soundness = do
  tie <- recallWith cvc5 tautologyImpliesEval
  nrt <- recall normalizeRespectsTruth
  nc  <- recall normalizeCorrect

  calc "soundness"
       (\(Forall f) (Forall bindings) -> isTautology f .=> eval f bindings) $
       \f bindings -> [isTautology f]
                   |- eval f bindings
                   ?? nrt `at` (Inst @"f" f, Inst @"bs" bindings)
                   =: eval (normalize f) bindings
                   ?? nc  `at` Inst @"f" f
                   ?? tie `at` (Inst @"f" (normalize f), Inst @"a" bindings, Inst @"b" [])
                   =: sTrue
                   =: qed
```

Completeness says that if the checker rejects a formula, a companion function `falsify` produces an assignment under which the formula is false:

```
completeness :: TP (Proof (Forall "f" Formula -> SBool))
completeness = do
  ch  <- recall completenessHelper
  nrt <- recallWith z3 normalizeRespectsTruth

  lemma "completeness"
        (\(Forall f) -> sNot (isTautology f) .=> sNot (eval f (scex (falsify f))))
        [proofOf ch, proofOf nrt]
```

Both top-level proofs are short because the work is in their lemmas. The development contains 21 TP lemmas. Several prove properties of variable lookup when assignments are extended; others

establish that normalization produces a normal formula and preserves meaning. The central inductive lemmas, such as `tautologyImpliesEval`, use *strong induction* via `sInduct`: the hypothesis is available for every formula with a smaller measure, not only for immediate subformulas, which matches the shape of the recursion in `isTautology'`. The lemmas use `recall` to reuse previously proved results and, in a few places, select CVC5 instead of Z3 for steps where it performs better. Soundness and completeness each check in one to two seconds.

What to take away. The development reuses every technique from the earlier sections at larger scale. Generalization appears again: the central lemma `tautologyImpliesEval` is stated for arbitrary variable assignments, not only for the empty list of assumptions with which `isTautology` starts. New techniques are explicit termination measures and strong induction. The structure of the proof follows the structure of the program: one group of lemmas per auxiliary function, and top-level theorems assembled from them by short calculations. SBV's documentation contains about forty further TP developments, including sorting algorithms, a semantics-preserving constant folder, and a compiler from expressions to a stack machine.

11 FROM VERIFIED CODE TO C

The symbolic programs we have written can also be compiled. SBV translates a symbolic function to C code that runs without a solver. For the saturating adder of Section 3:

```
generateAdder :: FilePath -> IO ()
generateAdder dir = compileToC (Just dir) "sat_add" $ do
  cgOverwriteFiles True
  cgGenerateDriver False
  x <- cgInput "x"
  y <- cgInput "y"
  cgReturn $ satAdd x y
```

This produces `sat_add.c` and a matching header. Because the C code is generated from the same `satAdd` that we proved correct, there is no manual rewrite in which the two could drift apart. Bit-vector operations are generated with explicit widths and signedness, so the code does not depend on C's integer promotion rules or on undefined signed overflow.

Code generation is not a verified compiler, and it is worth being precise about what is known. The proof establishes a property of the *symbolic* function. The C program is a separate artifact, produced by a translation that must implement each operation faithfully. For a function with a small input domain we can close the gap by testing: the companion compiles the generated adder with undefined-behavior sanitization and compares it on all 65,536 inputs with an independent oracle written directly in C. The proof and the exhaustive test make different claims, about the model and about the compiled code, and together they give confidence in both.

Not all of SBV's vocabulary is executable. Quantifiers and uninterpreted sorts, for example, have no operational meaning, and the code generator rejects programs that use them rather than approximating them. Uninterpreted functions become calls to C functions that the user provides, which then join the trusted code.

12 UNDER THE HOOD

This section explains enough of SBV's implementation to make its behavior predictable. Figure 1 shows the overall flow.

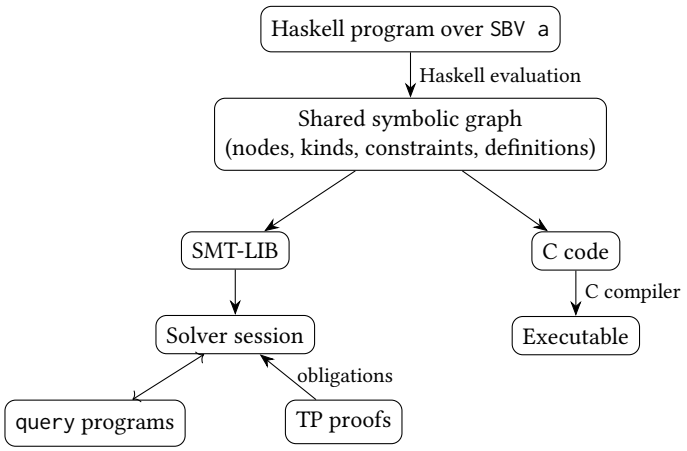


Fig. 1. Haskell evaluation builds a shared graph, which is translated to SMT-LIB for the solver or to C for execution. Queries and proofs drive the solver.

Symbolic values are deferred graph nodes. Internally, a value of type `SBV a` is either a concrete constant or a deferred computation that, given the state of the current symbolic run, adds a node to a graph and returns its identifier:

```
newtype SBV a = SBV SVal
data SVal     = SVal Kind (Either CV (Cached SV))
newtype Cached a = Cached (State -> IO a)
```

This representation lets `x + y` have the pure type `SWord8 -> SWord8 -> SWord8` while still allocating graph nodes. Operations on constants are folded immediately, which is why `satAdd 250 10` evaluates to `255` without a solver. The run-time `Kind` records the logical sort, so simplifications can respect it: rewriting `x .== x` to true is valid for integers but not for floats.

Sharing comes in two forms. A generator that combines a term with itself, such as `iterate (\t -> t + t) x !! 30`, describes a term that is exponentially large as a tree but linear as a graph. SBV preserves this sharing in two ways. It memoizes each deferred computation, using stable names in the style of observable sharing [Gill 2009], so that forcing the same Haskell value twice yields the same node. It also interns nodes, so that separately constructed applications of the same operator to the same arguments reuse a node. Neither mechanism identifies arbitrary mathematically equal expressions.

Nodes belong to a run. A graph node is meaningful only in the run that created it. Two values of type `SInteger` from different calls to `prove` cannot be combined, even though their types agree, and SBV checks for common mistakes of this kind. To carry a result from one run to another, extract a concrete value from a model and turn it back into a literal, as the query in Section 6 does with `literal v`.

Queries synchronize dependencies. When a query creates a new term, SBV records the declarations it requires: sorts for datatypes (including those nested inside lists, tuples, or other datatypes), uninterpreted constants, and definitions of `smtFunction`s it calls. Before the term is used, any of these that the solver has not seen are sent. This is why the datatype example in Section 7 works even when the datatype first appears inside a query.

13 PITFALLS AND WHAT TO TRUST

Common pitfalls.

- *Wrong domain.* A property proved over `SInteger` says nothing about `SInt32`, and vice versa. Choose the type that matches the code you care about.
- *Specification repeats the implementation.* Write specifications in a different style or a wider domain (Section 3).
- *Haskell* `if` *or recursion on symbolic data.* Use `ite` and `sCase` for symbolic choices, and `smtFunction` for recursion over symbolic structure (Section 5).
- *Contradictory assumptions.* If `constrained` assumptions are unsatisfiable, everything is provable. A quick `sat` on the assumptions alone detects this.
- *Treating unknown as an answer.* A timeout or unknown result is neither a proof nor a counterexample.
- *Too specific an induction statement.* If a proof step involving the induction hypothesis fails, the statement probably needs generalizing (Sections 8 and 9).
- *Mixing runs.* Symbolic values do not survive the run that created them; move concrete values instead (Section 12).

What a proof rests on. A Q.E.D. from SBV depends on the correctness of SBV's translation to SMT-LIB, the solver, and, for TP, the generation of proof obligations. TP proofs also depend on any axioms the user introduces and on any functions whose termination checks were explicitly bypassed. SBV tracks these dependencies: a proof that rests on an axiom, or on an unfinished step marked with `sorry`, is reported as `Modulo` the offending lemmas rather than as `Proven`. TP does not replay solver certificates in an independent proof kernel, so its guarantees are those of the underlying solver. Properties of symbolic lists are properties of finite lists: unlike Haskell lists, symbolic lists contain no bottoms and no infinite values. For generated C, the C compiler and the code generator join the trusted base, which is why Section 11 validates the generated code separately.

14 RELATED TOOLS AND FURTHER READING

QuickCheck [Claessen and Hughes 2000] and SBV ask similar questions with different evidence: a passing test campaign samples the input space, while an unsatisfiability result covers it. The two combine well, and SBV's own test suite compares concrete evaluation with solver results using QuickCheck.

Several systems embed solver-aided programming in a host language. Rosette [Torlak and Bodík 2014] provides a symbolic virtual machine for Racket that lifts ordinary host constructs to symbolic values. Grisette [Lu and Bodík 2023] is a Haskell library for symbolic compilation that focuses on efficient symbolic evaluation of programs with state merging. Smten [Uhler and Dave 2013] translated high-level Haskell-like symbolic computations into SMT queries. The what4 library [Galois, Inc. 2026] offers a typed Haskell interface to many solvers and is used inside verification tools. SBV makes the boundary between Haskell and symbolic control explicit through its types and operators, and adds the TP proof layer and C generation on the same representation.

For proofs about Haskell programs, Liquid Haskell [Vazou et al. 2014] checks refinement types with an SMT solver, and refinement reflection [Vazou et al. 2018] extends it to equational proofs about user-defined functions. There, specifications are refinement types on the program itself and proofs are Haskell terms checked against them; SBV's TP proofs are instead separate Haskell programs about symbolic models. Dafny [Leino 2010] shows how far the combination of user-supplied invariants and automatic verification-condition discharge can go in a dedicated language.

The SBV distribution contains a large collection of documented examples under `Documentation.SBV.Examples`, covering puzzles, cryptography, code generation, optimization, and theorem proving. The TP examples mentioned in Section 10 are a good next step after this tutorial.

15 CONCLUSION

SBV lets a Haskell programmer move gradually from checking a single arithmetic fact to proving properties of recursive algorithms, using one vocabulary of symbolic values throughout. The skills that matter change along the way: choosing the semantic domain, writing an independent specification, separating Haskell control from symbolic control, and, for inductive proofs, finding a statement general enough to prove. The solver handles the rest. We hope the examples here, all of which run, encourage readers to bring a solver into their own Haskell programs.

REFERENCES

Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2025. *The SMT-LIB Standard: Version 2.7.* Technical Report. Department of Computer Science, The University of Iowa. https://smt-lib.org/language.shtml

Jon Bentley. 1984. Programming Pearls: Algorithm Design Techniques. *Commun. ACM* 27, 9 (1984), 865–873. https://doi.org/10.1145/358234.381162

Robert S. Boyer and J Strother Moore. 1979. *A Computational Logic.* Academic Press, New York.

Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming.* ACM, New York, NY, USA, 268–279. https://doi.org/10.1145/351240.351266

Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems.* Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

Levent Erkök. 2026. SBV: SMT Based Verification in Haskell. Hackage package `sbv`. https://hackage.haskell.org/package/sbv

Galois, Inc. 2026. what4: Solver-agnostic symbolic values support for issuing queries. Hackage package `what4`. https://hackage.haskell.org/package/what4

Andy Gill. 2009. Type-Safe Observable Sharing in Haskell. In *Proceedings of the Second ACM SIGPLAN Symposium on Haskell.* ACM, New York, NY, USA, 117–128. https://doi.org/10.1145/1596638.1596653

K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (Lecture Notes in Computer Science, Vol. 6355).* Springer, Berlin, Heidelberg, 348–370. https://doi.org/10.1007/978-3-642-17511-4_20

Sirui Lu and Rastislav Bodík. 2023. Grisette: Symbolic Compilation as a Functional Programming Library. *Proceedings of the ACM on Programming Languages* 7, POPL, Article 16 (2023), 33 pages. https://doi.org/10.1145/3571209

Grant Passmore. 2025. Tautology Checker Example for ImandraX (`tautology.iml`). Imandra examples repository. https://github.com/imandra-ai/imandrax-examples/blob/main/src/tautology.iml

Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation.* ACM, New York, NY, USA, 530–541. https://doi.org/10.1145/2594291.2594340

Richard Uhler and Nirav Dave. 2013. Smten: Automatic Translation of High-Level Symbolic Computations into SMT Queries. In *Computer Aided Verification.* Springer, Berlin, Heidelberg, 678–683. https://doi.org/10.1007/978-3-642-39799-8_45

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. Refinement Types for Haskell. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming.* ACM, New York, NY, USA, 269–282. https://doi.org/10.1145/2628136.2628161

Niki Vazou, Anish Tondwalkar, Vikraman Choudhury, Ryan G. Scott, Ryan R. Newton, Philip Wadler, and Ranjit Jhala. 2018. Refinement Reflection: Complete Verification with SMT. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 53 (2018), 31 pages. https://doi.org/10.1145/3158141